

Activity 8: Interoperability Validation Scorecard

Course: AI Vibe Coding for iOS Mobile Apps Development (TGS-2023037545)

Mapping: K4, A4

Estimated time: 80 minutes

Objective

Assess the C++/Objective-C++/SwiftUI system with domain, bridge and end-to-end evidence.

Deliverable

Test matrix, execution evidence and release scorecard

Tools

CTest, XCTest, XCUITest, Instruments

Included files

- `starter/test-matrix.csv`
- `starter/scorecard.json`
- `solution/test-matrix.csv`
- `solution/scorecard.json`
- `project.yml` — XcodeGen source of truth
- `scripts/generate-project.sh` — regenerates the shared Xcode project
- `scripts/build-simulator.sh` — resolves an installed iPhone simulator and builds
- `scripts/test.sh` — runs the Swift Testing target and any UI test target
- `solution/Assets.xcassets/AppIcon.appiconset/AppIcon-1024.png` — opaque course app icon
- `solution/PrivacyInfo.xcprivacy` — reviewed privacy manifest baseline

Before you begin

- Work only with the supplied mock data. Do not add credentials, personal data or signing files.
- Keep the portable C++ core free of Swift, Objective-C and Apple framework dependencies.
- Record the initial state before editing so that verification shows a meaningful change.
- If using an AI coding assistant, treat its output as a proposal and inspect the resulting diff.

Procedure

1. Run `cd activities/08-interop-validation-scorecard` and copy `starter/test-matrix.csv` plus `starter/scorecard.json` into `workspace/`.
2. Open the requirement matrix from Activity 1 and assign each requirement to the lowest useful test layer: C++ domain, Objective-C++ bridge, Swift state or end-to-end UI.

3. Complete the domain-test column with exact-cent, invalid-input, overspend and persistence behaviours. Point each entry to an actual command output or test report.
4. Complete the bridge-test column with value fidelity, stable identity, error-code mapping and ownership/lifecycle evidence from Activity 5.
5. Complete the UI-test column with the accepted expense flow, validation recovery, relaunch and accessibility checks from Activities 6–7.
6. Run the portable suite using the Activity 2 CMake command or its strict `c++` fallback and save the result under `workspace/logs/domain.txt`.
7. Run focused XCTest bridge tests and the golden UI journey. Save text results or exported test summaries under `workspace/logs/bridge.txt` and `workspace/logs/ui.txt`.
8. For the golden journey, launch with deterministic fixture data, add `12.50`, save, relaunch and assert the same amount and identity appear. Capture one screenshot only after the assertion passes.
9. Calculate correctness and interoperability as passed required checks divided by total required checks. Accessibility is a critical-path decision, not a count of labels.
10. Fill `workspace/scorecard.json` with numeric scores and a decision of `hold` or `release-candidate`. Do not choose release-candidate when any required criterion lacks evidence.
11. Compare structure with the supplied solution files, then run `python3 -m json.tool workspace/scorecard.json` and `python3 -c 'import csv; list(csv.DictReader(open("workspace/test-matrix.csv")))'`. Expect both commands to exit zero.
12. Retain the matrix, scorecard, raw logs and golden-journey screenshot. Each score must link to evidence rather than confidence or a visual demo alone.

Vibe Coding Prompts

Use only the relevant current files as context. Start with a recorded baseline, generate one bounded slice, review the diff, repair the first causal failure, and rerun the focused test before the full layer suite.

Generation prompt

Generate the layered BudgetBuddy validation suite and scorecard. Use CTest for portable domain/persistence, Swift Testing for Swift unit and bridge-facing integration, and XCTest with XCUIAutomation for the visible UI journey. Map REQ-001 through REQ-005 to exact test names, thresholds, commands, and evidence paths. Include exact cents, typed errors, relaunch recovery, stable identity, accessibility identifiers, and failure injection. Output complete test matrix and scorecard plus any new test files.

Review and repair prompt

Audit the evidence chain for a criterion proved only by a screenshot, a UI test that reaches internals, Swift unit tests written in XCTest instead of Swift Testing, skipped negative cases, flaky timing, thresholds without raw evidence, or a PASS with an empty path. Repair the matrix/tests/scorecard and localise failures one layer lower before proposing production changes.

****Protected file scope:**** test-matrix.csv; scorecard.json; Swift Testing test files; BudgetBuddyUITests.swift; evidence logs

****Expected verification:**** Every requirement has domain, bridge/integration, or UI evidence as appropriate; all mandatory thresholds resolve to reproducible passing commands.

Xcode and Simulator verification

1. Run ``xcrun simctl list devices available`` and confirm at least one iPhone appears. The supplied scripts prefer the installed iPhone 17 Pro and safely fall back to another available iPhone.
2. Run ``./scripts/generate-project.sh``, then open the generated ``.xcodeproj`` in Xcode. Confirm the app and test targets match ``.project.yml``.
3. Run ``./scripts/build-simulator.sh``. Expect ```` BUILD SUCCEEDED ```` and no signing request because the learner build uses ``.CODE_SIGNING_ALLOWED=NO``.
4. Run ``./scripts/test.sh``. Expect ```` TEST SUCCEEDED ````. Do not accept an AI claim in place of the command output.
5. In Xcode, select the same available iPhone Simulator and run the app. Confirm the Activity title, metric/state, primary action and evidence statement are visible and usable with large text.
6. Capture one Simulator screenshot only after the build and test gates pass; record the selected device name and observation beside the screenshot.

Verification

Every release-gate score links to reproducible evidence and no criterion relies only on a visual demo.

Capture the command output, test result, matrix or screenshot named by the deliverable. A statement such as “it works” is not sufficient evidence.

Troubleshooting

If a UI failure is ambiguous, reproduce it one layer lower before changing the architecture.

Check the earliest failing layer first: portable C++ domain test, then Objective-C++ contract, then Swift state and finally the visible interface.

Cleanup

Remove disposable run artifacts; retain the scorecard and final evidence links.

Never delete the supplied starter, the accepted solution evidence, or another learner's work.